

Experiment

‘p_4_2_d_adjacent_duplicates_seq_without_4th_ensures’

Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures

Natural language statement: Write a method that takes a sequence of integers and returns all adjacent duplicates. For example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return [3, 5, 6].

Method signature: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates: seq<int>)

Ensures

- forall d :: d in duplicates ==> d in inputs
- forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates
- forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns all adjacent duplicates. For
→ example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return
→ [3, 5, 6].

The signature should be:

p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates
→ : seq<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates ==> d in inputs, ensures forall i :: 0 <= i < |inputs|
→ - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates, ensures forall d
→ :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i
→ + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn3woUWxjHvHs7nwMKF7aWTkt6y3E

Model: gpt-5.5-2026-04-23

Created at: 1780585618

Finish reason: stop

Usage: 538 tokens in, and 8785 tokens out

```
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
      → inputs[j] in duplicates
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
      → && inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases |inputs| - i
  {
    var oldI := i;
    ghost var oldDuplicates := duplicates;
```

```

assert forall d :: d in oldDuplications ==> d in inputs;
assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
  ↪ inputs[j] in oldDuplications;
assert forall d :: d in oldDuplications ==> exists j :: 0 <= j < oldI && j < |inputs| -
  ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

if inputs[i] == inputs[i + 1] {
  if !(inputs[i] in duplicates) {
    assert 0 <= i < |inputs|;
    assert inputs[i] in inputs;
    duplicates := duplicates + [inputs[i]];
    assert duplicates == oldDuplications + [inputs[oldI]];
    assert inputs[oldI] in duplicates;
  } else {
    assert inputs[oldI] in duplicates;
  }
}

assert duplicates == oldDuplications || duplicates == oldDuplications + [inputs[oldI]];
assert duplicates != oldDuplications ==> duplicates == oldDuplications + [inputs[oldI]];
assert duplicates != oldDuplications ==> inputs[oldI] == inputs[oldI + 1];
assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

assert forall d | d in oldDuplications
  ensures d in duplicates
{
  if duplicates != oldDuplications {
    assert duplicates == oldDuplications + [inputs[oldI]];
    assert d in oldDuplications + [inputs[oldI]];
  }
}

assert forall d | d in duplicates
  ensures d in inputs
{
  if d in oldDuplications {
    assert d in inputs;
  } else {
    assert duplicates != oldDuplications;
    assert duplicates == oldDuplications + [inputs[oldI]];
    assert d in oldDuplications + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert 0 <= oldI < |inputs|;
    assert inputs[oldI] in inputs;
  }
}

i := i + 1;
assert i == oldI + 1;

```

```

assert forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  if j < oldI {
    assert inputs[j] in oldDuplicates;
    assert inputs[j] in duplicates;
  } else {
    assert oldI <= j;
    assert j < oldI + 1;
    assert j == oldI;
    assert inputs[oldI] == inputs[oldI + 1];
    assert inputs[oldI] in duplicates;
  }
}

assert forall d | d in duplicates
  ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
    ↪ inputs[j] == d
{
  if d in oldDuplicates {
    assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
      ↪ && inputs[j] == d;
    ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
      ↪ inputs[k] == d;
    assert 0 <= k < i;
    assert k < |inputs| - 1;
    assert inputs[k] == inputs[k + 1];
    assert inputs[k] == d;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert inputs[oldI] == inputs[oldI + 1];
    assert 0 <= oldI < i;
    assert oldI < |inputs| - 1;
  }
}

assert forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  assert !(i < |inputs| - 1);
  assert j < i;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_4_2_d_adjacent_duplicates_seq_without_4th_ensures/program.dfy(64,6): Error
  ↪ : invalid AssertStmt
  |
64 | i := i + 1;
  | ^

1 parse errors detected in program.dfy
```

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn3zf24y94DLGZrW5s8Yc56hZAJYs

Model: gpt-5.5-2026-04-23

Created at: 1780585795

Finish reason: stop

Usage: 1743 tokens in, and 5850 tokens out

```
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
      ↪ inputs[j] in duplicates
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
      ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases |inputs| - i
  {
    var oldI := i;
    ghost var oldDuplicates := duplicates;

    assert forall d :: d in oldDuplicates ==> d in inputs;
    assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
      ↪ inputs[j] in oldDuplicates;
    assert forall d :: d in oldDuplicates ==> exists j :: 0 <= j < oldI && j < |inputs| -
      ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

    if inputs[i] == inputs[i + 1] {
      if !(inputs[i] in duplicates) {
        assert 0 <= i < |inputs|;
        assert inputs[i] in inputs;
        duplicates := duplicates + [inputs[i]];
        assert duplicates == oldDuplicates + [inputs[oldI]];
        assert inputs[oldI] in duplicates;
```

```

    } else {
      assert inputs[oldI] in duplicates;
    }
  }

  assert duplicates == oldDuplicates || duplicates == oldDuplicates + [inputs[oldI]];
  assert duplicates != oldDuplicates ==> duplicates == oldDuplicates + [inputs[oldI]];
  assert duplicates != oldDuplicates ==> inputs[oldI] == inputs[oldI + 1];
  assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

  forall d | d in oldDuplicates
    ensures d in duplicates
  {
    if duplicates != oldDuplicates {
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert d in oldDuplicates + [inputs[oldI]];
    }
  }

  forall d | d in duplicates
    ensures d in inputs
  {
    if d in oldDuplicates {
      assert d in inputs;
    } else {
      assert duplicates != oldDuplicates;
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert d in oldDuplicates + [inputs[oldI]];
      assert d in [inputs[oldI]];
      assert d == inputs[oldI];
      assert 0 <= oldI < |inputs|;
      assert inputs[oldI] in inputs;
    }
  }

  i := i + 1;
  assert i == oldI + 1;

  forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates
  {
    if j < oldI {
      assert inputs[j] in oldDuplicates;
      assert inputs[j] in duplicates;
    } else {
      assert oldI <= j;
      assert j < oldI + 1;
      assert j == oldI;
      assert inputs[oldI] == inputs[oldI + 1];
      assert inputs[oldI] in duplicates;
    }
  }

```

```

}

forall d | d in duplicates
  ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
    ↪ inputs[j] == d
{
  if d in oldDuplicates {
    assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
      ↪ && inputs[j] == d;
    ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
      ↪ inputs[k] == d;
    assert 0 <= k < i;
    assert k < |inputs| - 1;
    assert inputs[k] == inputs[k + 1];
    assert inputs[k] == d;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert inputs[oldI] == inputs[oldI + 1];
    assert 0 <= oldI < i;
    assert oldI < |inputs| - 1;
  }
}
}

forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  assert !(i < |inputs| - 1);
  assert j < i;
}
}

```

Final Program

```

method p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (
  ↪ duplicates: seq<int>)
  ensures forall d :: d in duplicates ==> d in inputs
  ensures forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs
    ↪ [i] in duplicates
  ensures forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 &&
    ↪ inputs[i] == inputs[i + 1] && inputs[i] == d
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|

```

```

invariant forall d :: d in duplicates ==> d in inputs
invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
  ↪ inputs[j] in duplicates
invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
  ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
decreases |inputs| - i
{
  var oldI := i;
  ghost var oldDuplicates := duplicates;

  assert forall d :: d in oldDuplicates ==> d in inputs;
  assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
    ↪ inputs[j] in oldDuplicates;
  assert forall d :: d in oldDuplicates ==> exists j :: 0 <= j < oldI && j < |inputs| -
    ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

  if inputs[i] == inputs[i + 1] {
    if !(inputs[i] in duplicates) {
      assert 0 <= i < |inputs|;
      assert inputs[i] in inputs;
      duplicates := duplicates + [inputs[i]];
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert inputs[oldI] in duplicates;
    } else {
      assert inputs[oldI] in duplicates;
    }
  }
}

assert duplicates == oldDuplicates || duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> inputs[oldI] == inputs[oldI + 1];
assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

forall d | d in oldDuplicates
  ensures d in duplicates
{
  if duplicates != oldDuplicates {
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
  }
}

forall d | d in duplicates
  ensures d in inputs
{
  if d in oldDuplicates {
    assert d in inputs;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
  }
}

```



```

    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert 0 <= oldI < |inputs|;
    assert inputs[oldI] in inputs;
  }
}

i := i + 1;
assert i == oldI + 1;

forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  if j < oldI {
    assert inputs[j] in oldDuplicates;
    assert inputs[j] in duplicates;
  } else {
    assert oldI <= j;
    assert j < oldI + 1;
    assert j == oldI;
    assert inputs[oldI] == inputs[oldI + 1];
    assert inputs[oldI] in duplicates;
  }
}

forall d | d in duplicates
  ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
    ↪ inputs[j] == d
{
  if d in oldDuplicates {
    assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
      ↪ && inputs[j] == d;
    ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
      ↪ inputs[k] == d;
    assert 0 <= k < i;
    assert k < |inputs| - 1;
    assert inputs[k] == inputs[k + 1];
    assert inputs[k] == d;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert inputs[oldI] == inputs[oldI + 1];
    assert 0 <= oldI < i;
    assert oldI < |inputs| - 1;
  }
}
}
}

```

```
forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  assert !(i < |inputs| - 1);
  assert j < i;
}
}
```

Total Token Usage

Input tokens: 2281

Output tokens: 14635

Reasoning tokens: 12108

Sum of ‘total tokens’: 16916

Experiment Timings

Overall Experiment started at 1780585616264, ended at 1780585943367, lasting 327103ms (327.10 seconds)

Iteration #1 started at 1780585616271, ended at 1780585794648, lasting 178377ms (178.38 seconds)

Iteration #2 started at 1780585794655, ended at 1780585943367, lasting 148712ms (148.71 seconds)